Errors

Sunday, 6 September 2026 1:29 PM

+ The Error Interface:-

Go programs express errors with error values. An error is any type that implements the simple builtin error interface.

```
type error interface {
        Error() string
}
```

- When something can go wrong in a function, that function should return an error as it's last return value. Any code that call a function that can return an error should handle errors by testing whether the error is nil.

- A nil error denotes success; a non-nil error denotes failure.

+ Formatting String Review.:

A convenient way to format strings in Go is by using the standard library's fmt.Sprintf() function it's a string interpolation function.

- The %v substring uses the type's default formatting which is often what you want

+ Because errors are just interfaces, you can build your own custom types that implement the error interface.

+ Keep in mind that the way Go handles errors is fairly unique. Most languages treat errors as something special and different. In Go, an error is just another value that we handle like any other value.

+ The Errors Package.:

The Go standard library provides an "errors" package that make it easy to deal with errors.

```
var err error = errors.New("Something went wrong")
```

+ Panic.:

There is another way to deal with errors in Go: the panic function. When a function call panic, the program crashes and prints a stack trace.

- As a general do not use panic

- use error values for all "normal" error handling. and if have an truly unrecoverable error, use log.Fatal to print a message and exit the program.

- The panic function yeets control out the current function and end up the call stact until it reaches a function that defer a recover. If no function call recover, the goroutine crashes.